

Documentazione CorsoService

Scopo della classe

CorsoService gestisce la logica di business relativa all'entità Corso. Estende AbstractService<Corso, CorsoDto>, ereditando le operazioni CRUD generiche e aggiungendo metodi di ricerca specifici per filtrare i corsi in base a vari criteri.

Dipendenze

- CorsoMapper: converte Entity Corso in CorsoDto e viceversa (lista o singolo oggetto).
- CorsoRepository: comunica con il database tramite JPA e contiene le query personalizzate relative ai corsi.

Costruttore

Riceve tramite Dependency Injection il repository generico, il converter, il mapper e il repository specifico (CorsoRepository). Il costruttore richiama `super(repository, converter)` per inizializzare la superclasse AbstractService e salva le dipendenze locali (corsoMapper e corsoRepository).

Metodi

trovaPerSport(String sport)

Ricerca tutti i corsi appartenenti a una determinata disciplina sportiva, ignorando la distinzione tra maiuscole e minuscole (case-insensitive). Il repository esegue la query `findBySportIgnoreCase(sport)`, il mapper converte la lista di Entity in una lista di CorsoDto e il metodo la restituisce.

trovaPerGiorno(GiorniEnum giorno)

Restituisce tutti i corsi erogati in uno specifico giorno della settimana indicato dall'enumerazione GiorniEnum. Il repository interroga la base dati con `findByGiorni(giorno)` e la lista ottenuta viene convertita in DTO tramite il mapper.

trovaDaOra(LocalTime ora)

Recupera tutti i corsi la cui ora di inizio è maggiore o uguale all'orario fornito in input (`findByOraInizioGreaterThanEqual(ora)`). I risultati vengono convertiti dal mapper in CorsoDto e restituiti al chiamante.

ottieniCorsiPerCampo(Integer campoid)

Restituisce tutti i corsi associati a uno specifico campo sportivo tramite il suo identificativo univoco (`findByCampoid(campoid)`). Le entità recuperate dal database vengono convertite in CorsoDto prima della restituzione.

Flusso di esecuzione

Controller → CorsoService → CorsoRepository → Database → List<Corso> → CorsoMapper → List<CorsoDto> → Controller → JSON.

Vantaggi

Separazione delle responsabilità, riutilizzo del codice CRUD tramite `AbstractService`, conversione Entity/DTO centralizzata e sicura, nessuna esposizione diretta del modello di persistenza, codice più pulito, isolato e facilmente manutenibile.